

## 5.6 ESP32 Dual-Board WIFI

### ESP32 Dual-Board WiFi TCP

If you are using one ESP32 and one UNO R4 WIFI, please refer to the wiring tutorial below.

<https://wiki.elegoo.com/en/oshw-getting-started-&-kits/simpleremote>

#### What You Can Build

After completing this project, you will have a dual ESP32 system that can chat bidirectionally via WiFi TCP protocol:

- ▶  **Two ESP32 development boards:** One acts as a TCP server (creates WiFi hotspot), the other as a TCP client (connects to the hotspot)
- ▶  **Bidirectional serial chat:** Type a message on either board's serial monitor, and the other receives it
- ▶  **Real-time bidirectional communication:** Both sides can send and receive messages simultaneously, just like chatting on WeChat
- ▶  **Status indicator:** LED shows connection status (on = connected, off = not connected)
- ▶  **Auto-reconnect:** Client automatically attempts to reconnect every 3 seconds when disconnected

**Simple analogy:** It's like two people chatting on WeChat — once the two ESP32 boards establish a WiFi TCP connection, type a message in either side's serial monitor and press Enter, the other side receives it instantly.

#### Step 1: Preparation

##### Communication Flow

1. **Server Startup:** Creates WiFi hotspot `ESP32_Server`, starts TCP server listening on port 8080
2. **Client Connects WiFi:** Connects to the hotspot created by Server, obtains IP address
3. **Client Establishes TCP Connection:** Connects to `192.168.4.1:8080`
4. **Server Welcome:** Sends welcome message to Client
5. **Bidirectional Chat:**
  - ▶ Type a message in Server's serial monitor and press Enter → Client receives it
  - ▶ Type a message in Client's serial monitor and press Enter → Server receives it
  - ▶ Both serial ports display `Sent:` and `Received:` markers
6. **Disconnect & Reconnect:** Client attempts to reconnect every 3 seconds after detecting disconnection

#### Step 2: Software Configuration and Code Flashing

 [DOWNLOAD](#)

##### Step 2.1: Open Project Code

1. Place the entire project folder ( `ESP32_Dual_Board_WiFi_TCP` ) on your computer in an easily accessible location (e.g., desktop)
2. You will see two subfolders, each containing an `.ino` file:
  - ▶ `ESP32_TCP_Server/ESP32_TCP_Server.ino` → Flash to Server board
  - ▶ `ESP32_TCP_Client/ESP32_TCP_Client.ino` → Flash to Client board

##### Step 2.2: Modify WiFi Configuration (Optional)



 **Default configuration works out of the box!** If you want to customize hotspot name and password, you need to modify both files simultaneously.

**Modify Server Side** ( `ESP32_TCP_Server.ino` ):

```
1 | const char* ap_ssid = "ESP32_Server"; // ⚠ Change to your desired hotspot name
2 | const char* ap_password = "12345678"; // ⚠ Change to your desired password (min 8 chars)
3 | const uint16_t serverPort = 8080; // TCP port
```

```

1 | const char* serverSSID = "ESP32_Server";    // ⚠ Must match Server side!
2 | const char* serverPassword = "12345678";    // ⚠ Must match Server side!
3 | const char* serverIP = "192.168.4.1";      // Server's AP IP, usually no need to change
4 | const uint16_t serverPort = 8080;          // ⚠ Must match Server side!

```



⚠ **Important!** If you change the hotspot name or password, **both sides must be modified simultaneously**, otherwise the Client cannot connect to the Server!

### Step 2.3: Flash Server Code

1. Connect **ESP32 Server** to your computer with a USB cable
2. Open Arduino IDE, click **File** → **Open** → select [ESP32\\_TCP\\_Server/ESP32\\_TCP\\_Server.ino](#)
3. Click **Tools** → **Board** → select your ESP32 board (e.g., [ESP32 Dev Module](#) )
4. Click **Tools** → **Port** → select the COM port for Server
5. Click the **Upload** button (arrow icon) to start flashing
6. Wait for flashing to complete, seeing [Hard resetting via RTS pin...](#) indicates success

### Step 2.4: Flash Client Code

1. Connect **ESP32 Client** to your computer with another USB cable
2. Open another Arduino IDE window (or close the previous one and reopen)
3. Click **File** → **Open** → select [ESP32\\_TCP\\_Client/ESP32\\_TCP\\_Client.ino](#)
4. Click **Tools** → **Board** → select your ESP32 board
5. Click **Tools** → **Port** → select the COM port for Client
6. Click the **Upload** button to start flashing

### Step 2.5: Observe Startup Process (Serial Monitor)

After successful flashing, let's see what the ESP32 is doing through the "Serial Monitor":

1. Open the serial monitor in both Arduino IDE windows ( [Ctrl+Shift+M](#) )
2. In the bottom-right corner of the serial monitor window, set the **baud rate** to **115200** (very important! Otherwise you'll see garbled text)
3. ⚠ **Critical Setting:** In the bottom-right corner of the serial monitor, set the **line ending** to **"Newline"** or **"Both NL & CR"**, otherwise the ESP32 may not correctly detect input completion!
4. Press the **EN button** (reset button) on both ESP32 boards
5. Observe the output in the serial monitor and compare with the "Expected Output" in the next section

## ✅ Step 3: Check If It Works Correctly

### 3.1 Server Side Expected Serial Output

```

1 | =====
2 | ESP32 TCP Server — Starting up...
3 | =====
4 | [SERVER] Creating AP hotspot: ESP32_Server
5 | [SERVER] AP IP Address: 192.168.4.1
6 | [SERVER] TCP Server started on port: 8080
7 | [SERVER] Waiting for client connection...
8 |
9 | [SERVER] Type a message and press Enter to send.

```

- ▶ Seeing [AP IP Address: 192.168.4.1](#) → WiFi hotspot created successfully ✓
- ▶ Seeing [TCP Server started on port: 8080](#) → TCP server started successfully ✓
- ▶ Waiting for Client to connect...

### 3.2 Client Side Expected Serial Output

```

1 | =====
2 | ESP32 TCP Client — Starting up...
3 | =====
4 | [CLIENT] Connecting to AP: ESP32_Server

```

```

5 | .....
6 | [CLIENT] WiFi connected!
7 | [CLIENT] Client IP: 192.168.4.2
8 | [CLIENT] Server IP: 192.168.4.1
9 | [CLIENT] Connecting to TCP server... CONNECTED!
10 | [CLIENT] Ready to chat. Type a message and press Enter.
11 | [CLIENT] Server says: [Server] Client connected! Ready to chat.

```

[Back to Elegoo](#)

- ▶ Seeing [WiFi connected!](#) → WiFi connection successful ✓
- ▶ Seeing [CONNECTED!](#) → TCP connection successful ✓
- ▶ Seeing [Server says: \[Server\] Client connected! Ready to chat.](#) → Received server welcome message ✓

### 3.3 Bidirectional Chat Effect

Type a message in Server's serial port, press Enter to send:

```

1 | [Server] Hello Client! How are you?
2 | [SERVER] Sent: Hello Client! How are you?

```

Client's serial port will receive:

```

1 | [CLIENT] Received: Hello Client! How are you?

```

Type a message in Client's serial port, press Enter to send:

```

1 | [CLIENT] Sent: I'm great! Thanks for asking!

```

Server's serial port will receive:

```

1 | [SERVER] Received: I'm great! Thanks for asking!

```

### 3.4 LED Status Indicator

| LED Location     | Status | Meaning                       |
|------------------|--------|-------------------------------|
| Server GPIO2 LED | Off    | Waiting for client connection |
| Server GPIO2 LED | On     | Client connected              |
| Client GPIO2 LED | Off    | Not connected to server       |
| Client GPIO2 LED | On     | Connected to server           |

### 3.5 Disconnect & Reconnect Test

Unplug the Client's USB cable (or press EN button to reset), Server will display:

```

1 | -----
2 | [SERVER] <<< Client disconnected.
3 | [SERVER] Waiting for next client...
4 | -----

```

Reconnect the Client, it will automatically reconnect and resume communication.

## 🤔 Step 4: Troubleshooting



💡 **Beginners look here first:** 90% of issues are listed below. If your situation is not in the list, go back and check all wiring and configuration from scratch.

**Q1: Serial monitor shows garbled text or blank?**

**A:** This is the most common beginner error, almost 100% caused by incorrect baud rate.

[Back to Elegoo](#)

- Check the baud rate in the bottom-right corner of the serial monitor, **it must be 115200**
- After changing the baud rate, press the EN (reset) button on the ESP32 to restart

## Q2: No response after typing a message in serial port?

**A:** The most likely cause is incorrect line ending setting in the serial monitor.

1. Check the **line ending setting** in the bottom-right corner of the serial monitor
2. Must be set to "**Newline**" or "**Both NL & CR**"
3. If set to "No line ending", the ESP32 cannot receive the complete message
4. Re-enter the message and press Enter to test

## Q3: Client keeps printing "Connecting to AP" but cannot connect?

**A:** Check the following in order:

1. **Confirm Server code is flashed and running:**
  - Check if Server serial shows [\[SERVER\] TCP Server started on port: 8080](#)
  - Use your phone to search for WiFi, see if you can find the [ESP32\\_Server](#) hotspot
2. **Check hotspot name and password:**
  - Confirm [serverSSID](#) and [serverPassword](#) in Client code match the Server side
  - Case-sensitive! [esp32\\_server](#) and [ESP32\\_Server](#) are different
  - Any extra spaces?
3. **Check password length:**
  - Hotspot password must be at least 8 characters, fewer will cause hotspot creation to silently fail
4. **Check WiFi signal strength:**
  - Too far apart? Move them closer (recommended within 5 meters)
  - Any metal obstacles between the two boards?

## Q4: Client connects to WiFi but TCP connection fails?

**A:** Check the following in order:

1. **Confirm Server's TCP Server is started:**
  - Server serial should show [\[SERVER\] TCP Server started on port: 8080](#)
2. **Check Server IP:**
  - [serverIP](#) in Client code should be [192.168.4.1](#) (Server's AP default IP)
  - If you modified Server's AP configuration, confirm the IP has not changed
3. **Check port number:**
  - [serverPort](#) on both sides must be consistent, default is [8080](#)
  - If you modified the port, change both sides

## Q5: Server cannot receive messages from Client?

**A:** Check one by one:

1. **Does Client show "CONNECTED!":**
  - If not, TCP connection is not established (go back to Q4)
2. **Can Client send messages:**
  - Type a message in Client's serial port, see if it shows [\[CLIENT\] Sent: ...](#)
  - If not shown, possibly a serial input issue (go back to Q2)
3. **Is Server listening:**
  - Server serial should show "New client connected!"
  - If not, Client's connection request did not reach Server

## Q6: Client cannot receive messages from Server?

[Back to Elegoo](#)

A: Check one by one:

### 1. Confirm Client is connected:

- Client serial should show [CONNECTED!](#) and [Ready to chat](#)

### 2. Is Server sending:

- Server serial should show [\[SERVER\] Sent: ...](#)
- If not shown, possibly a serial input issue (go back to Q2)

### 3. Is Client receiving:

- Client serial should show [\[CLIENT\] Received: ...](#)
- If not, possibly the connection is broken

## Q7: Client LED not on or Server LED not on?

A: Check the following:

### 1. LED pin definition:

- Server side: [LED\\_PIN = 2](#) (GPIO2)
- Client side: [LED\\_PIN = 2](#) (GPIO2)
- GPIO2 is the onboard LED pin on most ESP32 development boards

### 2. Confirm LED status logic:

- Server LED only turns on after Client connects
- Client LED only turns on after connecting to Server
- Please first ensure TCP connection is successfully established

## Q8: Client auto-reconnect not working?

A: Check the following in order:

### 1. Confirm Client detected the disconnection:

- Client serial should show [\[CLIENT\] Attempting to reconnect...](#)

### 2. Check reconnect interval:

- The reconnect interval is set to 3 seconds in the code
- If you need faster or slower reconnection, modify the [3000](#) value

### 3. Confirm Server is running:

- Before reconnecting, Server must be running and the hotspot must be broadcasting normally

## Q9: Code changes not taking effect?

A: Check the following:

### 1. Did you re-flash the code:

- Code changes must be re-uploaded to the development board

### 2. Did you modify the correct file:

- Server function modifications → [ESP32\\_TCP\\_Server.ino](#)
- Client function modifications → [ESP32\\_TCP\\_Client.ino](#)

### 3. Did you modify both sides simultaneously:

- If modifying SSID, password, port, etc., both sides must be changed

## Troubleshooting Quick Reference Table (Quick Positioning)

| Problem Symptom         | Most Likely Cause         | Fastest Solution                      |
|-------------------------|---------------------------|---------------------------------------|
| Serial garbled / blank  | Incorrect baud rate       | Change to 115200, press EN to reset   |
| No response after input | Line ending setting wrong | Change to "Newline" or "Both NL & CR" |

| Problem Symptom            | Most Likely Cause               | Fastest Solution                                          |
|----------------------------|---------------------------------|-----------------------------------------------------------|
| Client cannot connect WiFi | Wrong password / weak signal    | Check SSID and password, move boards closer               |
| TCP connection fails       | Wrong Server IP/port            | Confirm <code>serverIP = 192.168.4.1</code> , ports match |
| Cannot receive messages    | Line ending / connection broken | Check line ending setting, reconnect                      |
| LED not on                 | Wrong GPIO definition           | Confirm <code>LED_PIN = 2</code>                          |
| Changes not taking effect  | Not re-flashed                  | Re-upload code                                            |
| Reconnect not working      | Server not running              | Confirm Server is running                                 |

[Back to Elegoo](#)

## 📖 Step 5: Understanding Project Principles



💡 After completing the experiment, understanding the principles behind it will yield twice the result with half the effort! This step is "knowing the what, and more importantly, the why".

### 5.1 Project Architecture: Server and Client Roles

This project uses a **dual-board architecture**, with two ESP32 boards taking on different roles:

| Device       | Role                 | Main Responsibilities                                              | Analogy                                                           |
|--------------|----------------------|--------------------------------------------------------------------|-------------------------------------------------------------------|
| ESP32 Server | Hotspot + TCP Server | Create WiFi, listen on port, receive messages, send messages       | Chat server (waiting for connections, sending/receiving messages) |
| ESP32 Client | TCP Client           | Connect to hotspot, establish TCP, send messages, receive messages | Chat client (actively connecting, sending/receiving messages)     |

#### Module Relationship Diagram:

|   | ESP32 Server (.ino)                 | ESP32 Client (.ino)                     |
|---|-------------------------------------|-----------------------------------------|
| 1 | —— WiFi.softAP() Create hotspot     | —— WiFi.begin() Connect to hotspot      |
| 2 | —— WiFiServer(8080) TCP Server      | —— WiFiClient.connect() TCP Client      |
| 3 | —— server.available() Wait for conn | —— Serial.available() Read serial input |
| 4 | —— Serial.available() Read serial   | —— client.println() Send message        |
| 5 | —— client.println() Send message    | —— client.readStringUntil() Receive     |
| 6 | —— client.readStringUntil() Receive | —— Auto-reconnect mechanism             |
| 7 |                                     |                                         |

#### Why use dual-board architecture?

| Advantage           | Explanation                                        |
|---------------------|----------------------------------------------------|
| Learn communication | Intuitive understanding of TCP client-server model |
| Independent testing | Debug Server and Client code separately            |
| Scalable            | Easy to add more clients later                     |
| Real-world scenario | Simulate real IoT device communication patterns    |

### 5.2 Why TCP Instead of UDP?

**TCP (Transmission Control Protocol)** is a connection-oriented reliable transport protocol with the following characteristics:

| Feature                      | Description                                    | Implementation in This Project                           |
|------------------------------|------------------------------------------------|----------------------------------------------------------|
| <b>Connection-oriented</b>   | Must establish connection before communicating | Client must first <code>connect()</code> to Server       |
| <b>Reliable transmission</b> | Guarantees no data loss or duplication         | TCP protocol itself guarantees reliable message delivery |
| <b>Ordering</b>              | Data arrives in send order                     | Messages arrive at the other end in order                |

| Feature            | Description                           | Implementation in This Project     |
|--------------------|---------------------------------------|------------------------------------|
| Flow control       | Prevents sender from sending too fast | Automatically handled by TCP stack |
| Congestion control | Prevents network overload             | Automatically handled by TCP stack |

#### UDP's shortcomings:

- UDP is a connectionless protocol, does not guarantee message delivery
- May cause message loss or reordering
- Suitable for scenarios requiring real-time performance but tolerating small losses (e.g., video streaming)



🔥 **Why choose TCP?** Because we need to ensure reliable message delivery. If using UDP, important messages might be lost, while TCP ensures every message can be received by the other party through connection mechanism and retransmission.

### 5.3 Server's Working Mode: Why Use AP Mode?

#### WiFi Mode Comparison:

| Mode        | Description                               | Used in This Project                                 |
|-------------|-------------------------------------------|------------------------------------------------------|
| STA Mode    | Act as client connecting to router        | Client uses this mode to connect to Server's hotspot |
| AP Mode     | Act as hotspot connected by other devices | Server uses this mode to create hotspot              |
| STA+AP Mode | Simultaneously act as client and hotspot  | ESP32 supports it but not needed in this project     |

#### Why Server uses AP mode instead of STA mode?

- **Independent operation:** Does not depend on external routers, can be used in environments without WiFi
- **Simplified configuration:** Client only needs to know Server's hotspot name and password, no router information needed
- **Improved reliability:** Not affected by external network failures
- **Suitable for teaching:** Demonstrates ESP32's AP functionality

### 5.4 Non-Blocking Programming: Why loop() Doesn't Get Stuck?

This is a very important concept in embedded programming — **non-blocking programming**.

✗ **Problem with `delay()`** : The program "gets stuck" here, unable to do anything else during that time.

✓ **Benefit of `millis()`** : `millis()` returns "how many milliseconds have elapsed since power-on". We use it to determine "is it time yet?" and continue doing other things if not.

#### Non-blocking main loop in Client:

```

1 | void loop() {
2 |   //1. Check connection status (non-blocking)
3 |   if (!clientConnected || !client.connected()) {
4 |     // Disconnect and reconnect logic...
5 |     return;
6 |   }
7 |
8 |   // 2. Read serial input and send (non-blocking)
9 |   if (Serial.available()) {
10 |    // Read and send if input exists
11 |   }
12 |
13 |   // 3. Read messages from other end (non-blocking)
14 |   if (client.available()) {
15 |    // Display if message exists
16 |   }
17 |
18 |   delay(10); // Short delay, does not affect responsiveness
19 | }
20 | // Each loop checks all conditions without being stuck by any one

```

#### Non-blocking listening in Server:

```
1 void loop(){
2   //1. Check new connections (non-blocking)
3   if (!clientConnected){
4     WiFiClient newClient = server.available();
5     if (newClient){ /* Handle new connection */
6     }
7
8     // 2. Read serial input and send (non-blocking)
9     if (Serial.available()){
10      // Read and send if input exists
11    }
12
13    // 3. Read messages from other end (non-blocking)
14    if (client.available()){
15      // Display if message exists
16    }
17
18    delay(10);
19  }
```

 **Key Concepts:**

- ▶ `Serial.available()` checks for serial data, reads if available, skips if not
- ▶ `client.available()` checks for TCP data, reads if available, skips if not
- ▶ `server.available()` checks for new connections, accepts if available, skips if not
- ▶ All these checks are **non-blocking** and will not get stuck in the main loop

### 5.5 TCP Communication Reliability Mechanism

This project ensures reliable message transmission through the following mechanisms:

| Mechanism                             | Implementation                                                    | Description                                                            |
|---------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------|
| Connection confirmation               | <code>client.connect()</code> return value                        | Only start sending messages after successful connection                |
| TCP reliable transmission             | TCP stack automatic handling                                      | Automatic retransmission on message loss, guaranteed reliable delivery |
| Timeout reconnection                  | <code>millis()</code> + timing judgment                           | Attempts to reconnect every 3 seconds after disconnection              |
| Real-time bidirectional communication | <code>Serial.available()</code> + <code>client.available()</code> | Both ends send and receive independently without blocking              |

### 5.6 Serial Input Processing Mechanism

The core of this project is **serial-input-driven bidirectional chat**, working as follows:

| Step                                                        | Description                                                |
|-------------------------------------------------------------|------------------------------------------------------------|
| 1. User types text in serial monitor                        | Ends with newline character (requires line ending setting) |
| 2. ESP32 detects via <code>Serial.available()</code>        | Checks if new serial data is available                     |
| 3. Reads via <code>Serial.readStringUntil("\n")</code>      | Reads until newline is encountered                         |
| 4. Sends via TCP <code>client.println()</code>              | Sends to other party, auto-adds newline                    |
| 5. Other party receives via <code>client.available()</code> | Checks if new TCP data is available                        |
| 6. Reads via <code>client.readStringUntil("\n")</code>      | Reads complete message                                     |
| 7. Displays via <code>Serial.println()</code>               | Displays on serial monitor                                 |

 **Important:** The serial monitor's **line ending setting** must be correct, otherwise the ESP32 cannot recognize message completion. Recommended setting is **"Newline"** or **"Both NL & CR"**.

## 6.1 Libraries and Key Functions Used in the Project



Before looking at specific code, learn about what libraries and functions this project uses, making it easier to read the code later.

## 6.1.1 Library Introduction

| Library Name           | Source              | Function Description                                                       |
|------------------------|---------------------|----------------------------------------------------------------------------|
| <a href="#">WiFi.h</a> | ESP32 Core Built-in | ESP32-specific WiFi library, provides AP/STA mode, TCP server/client, etc. |

## 6.1.2 WiFi Related Functions

| Function/Command                        | Meaning and Purpose                                                                   |
|-----------------------------------------|---------------------------------------------------------------------------------------|
| <a href="#">WiFi.mode(WIFI_AP)</a>      | Set WiFi to <b>Hotspot Mode</b> . ESP32 itself becomes a WiFi hotspot                 |
| <a href="#">WiFi.mode(WIFI_STA)</a>     | Set WiFi to <b>Station Mode</b> . ESP32 acts as client connecting to other hotspots   |
| <a href="#">WiFi.softAP(ssid, pass)</a> | <b>Create AP hotspot</b> . Pass in hotspot name and password                          |
| <a href="#">WiFi.softAPIP()</a>         | <b>Get IP address in AP mode</b> . Returns <a href="#">192.168.4.1</a> by default     |
| <a href="#">WiFi.begin(ssid, pass)</a>  | <b>Start WiFi connection</b> . Connect to specified hotspot                           |
| <a href="#">WiFi.status()</a>           | <b>Query current connection status</b> . <a href="#">WL_CONNECTED</a> means connected |
| <a href="#">WiFi.disconnect()</a>       | <b>Disconnect WiFi connection</b>                                                     |
| <a href="#">WiFi.localIP()</a>          | <b>Get IP address in STA mode</b>                                                     |

## 6.1.3 TCP Server Related Functions

| Function/Command                             | Meaning and Purpose                                                                                            |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <a href="#">WiFiServer server(port)</a>      | <b>Create TCP server instance</b> . Specify listen port (default 8080)                                         |
| <a href="#">server.begin()</a>               | <b>Start TCP server</b> . Start listening on port, accept client connections                                   |
| <a href="#">server.available()</a>           | <b>Non-blocking check for new connections</b> . Returns <a href="#">WiFiClient</a> object when client connects |
| <a href="#">client.remoteIP()</a>            | <b>Get client IP address</b>                                                                                   |
| <a href="#">client.println(data)</a>         | <b>Send data to client</b> . Auto-adds newline                                                                 |
| <a href="#">client.readStringUntil('\n')</a> | <b>Read data from client</b> . Reads until newline                                                             |
| <a href="#">client.available()</a>           | <b>Check if data can be read</b>                                                                               |
| <a href="#">client.connected()</a>           | <b>Check if connection is still valid</b>                                                                      |
| <a href="#">client.stop()</a>                | <b>Close client connection</b>                                                                                 |

## 6.1.4 TCP Client Related Functions

| Function/Command                         | Meaning and Purpose                                                     |
|------------------------------------------|-------------------------------------------------------------------------|
| <a href="#">WiFiClient client</a>        | <b>Create TCP client instance</b>                                       |
| <a href="#">client.connect(ip, port)</a> | <b>Connect to TCP server</b> . Returns <a href="#">true</a> for success |
| <a href="#">client.println(data)</a>     | <b>Send data to server</b> . Auto-adds newline                          |

| Function/Command                          | Meaning and Purpose                |
|-------------------------------------------|------------------------------------|
| <code>client.readStringUntil('\n')</code> | Read data from server              |
| <code>client.available()</code>           | Check if data can be read          |
| <code>client.connected()</code>           | Check if connection is still valid |
| <code>client.stop()</code>                | Close TCP connection               |

6.1.5 Arduino Core Functions

| Function/Command                                            | Meaning and Purpose                                                         |
|-------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>pinMode(pin, OUTPUT)</code>                           | Configure pin as output mode. Used for LED control                          |
| <code>digitalWrite(pin, HIGH/LOW)</code>                    | Set pin output voltage. <code>HIGH</code> = 3.3V, <code>LOW</code> = 0V     |
| <code>millis()</code>                                       | Return milliseconds since boot. Used for non-blocking timing                |
| <code>delay(ms)</code>                                      | Delay specified milliseconds. ⚠ Blocks program, only used in initialization |
| <code>Serial.begin(115200)</code>                           | Initialize serial communication. Baud rate 115200                           |
| <code>Serial.available()</code>                             | Check if serial data can be read. Non-blocking, returns 0 or 1              |
| <code>Serial.readStringUntil('\n')</code>                   | Read string from serial port. Reads until newline                           |
| <code>Serial.print()</code> / <code>Serial.println()</code> | Print information to serial port. Used for debug output                     |

6.2 Server Code Analysis: ESP32\_TCP\_Server.ino



Server is responsible for creating WiFi hotspot and TCP server, waiting for Client connection and handling bidirectional chat.

6.2.1 Configuration Section

```
1  #include <WiFi.h>
2
3  //===== Configuration =====
4  const char* ap_ssid  = "ESP32_Server"; // Hotspot name
5  const char* ap_password = "12345678"; // Hotspot password (min 8 chars)
6  const uint16_t serverPort = 8080; // TCP server port
7
8  //===== LED =====
9  #define LED_PIN 2 // Onboard LED pin
10
11 //===== TCP Server =====
12 WiFiServer server(serverPort); // Create TCP server instance
13 WiFiClient client; // Persistent client object
14 bool clientConnected = false; // Connection status flag
```

**Running Logic:** Include WiFi library, define hotspot info, LED pin, TCP server instance, and status variables.

**Key Command Analysis:**

- ▶ `const char* ap_password = "12345678"` : Password must be **at least 8 characters**, fewer will cause `softAP()` to silently fail
- ▶ `WiFiServer server(8080)` : Create TCP server object, listening on port 8080
- ▶ `WiFiClient client` : **Persistent** client object, used across multiple calls in `loop()` to maintain connection
- ▶ `clientConnected` : Connection status flag, controls whether to process messages

6.2.2 `setup()`: Initialization Function (runs only once)

```
1  void setup() {
2    Serial.begin(115200);
3    delay(1000);
4  }
```

```

4
5 pinMode(LED_PIN, OUTPUT);
6 digitalWrite(LED_PIN, LOW);
7
8 Serial.println();
9 Serial.println("=====");
10 Serial.println(" ESP32 TCP Server — Starting up...");
11 Serial.println("=====");
12
13 // Step 1: Create WiFi hotspot
14 Serial.print("[SERVER] Creating AP hotspot: ");
15 Serial.println(ap_ssid);
16 WiFi.mode(WIFI_AP);
17 WiFi.softAP(ap_ssid, ap_password);
18 delay(1000);
19
20 Serial.print("[SERVER] AP IP Address: ");
21 Serial.println(WiFi.softAPIP());
22
23 // Step 2: Start TCP server
24 server.begin();
25 Serial.print("[SERVER] TCP Server started on port: ");
26 Serial.println(serverPort);
27 Serial.println("[SERVER] Waiting for client connection...");
28 Serial.println();
29 Serial.println("[SERVER] Type a message and press Enter to send.");
30 Serial.println();
31 }

```

**Running Logic:** After power-on, sequentially completes 5 tasks:

1. Initialize serial port and LED
2. Create WiFi hotspot
3. Start TCP server
4. Print startup information
5. Prompt user to start chatting

#### Key Command Analysis:

- `WiFi.mode(WIFI_AP)` : Switch to AP mode, ESP32 becomes WiFi hotspot
- `WiFi.softAP(ap_ssid, ap_password)` : Create hotspot, Client can search and connect
- `WiFi.softAPIP()` : Get IP address in AP mode (default `192.168.4.1`)
- `server.begin()` : Start TCP server, begin listening on port

#### 6.2.3 loop(): Main Loop (bidirectional chat handling)

```

1 void loop() {
2   // Check for incoming client connections (non-blocking)
3   if (!clientConnected) {
4     WiFiClient newClient = server.available();
5     if (newClient) {
6       client = newClient;
7       clientConnected = true;
8       digitalWrite(LED_PIN, HIGH);
9       Serial.println("-----");
10      Serial.println("[SERVER] >>> New client connected!");
11      Serial.print("[SERVER] Client IP: ");
12      Serial.println(client.remoteIP());
13
14      // Send welcome message to client
15      client.println("[Server] Client connected! Ready to chat.");
16      Serial.println("[SERVER] Ready to chat. Type a message and press Enter.");
17      Serial.println("-----");
18    }
19  }
20
21  // When connected, handle bidirectional communication
22  if (clientConnected) {
23    // Check if client disconnected
24    if (!client.connected()) {
25      clientConnected = false;
26      digitalWrite(LED_PIN, LOW);
27    }

```

```

27 client.stop();
28 Serial.println("-----");
29 Serial.println("[SERVER] <<< Client disconnected.");
30 Serial.println("[SERVER] Waiting for next client...");
31 Serial.println("-----");
32 return;
33 }
34
35 // 1. Read from serial input and send to client
36 if (Serial.available()) {
37   String msg = Serial.readStringUntil('\n');
38   msg.trim();
39   if (msg.length() > 0) {
40     client.println(msg);
41     Serial.print("[SERVER] Sent: ");
42     Serial.println(msg);
43   }
44 }
45
46 // 2. Read from client and display
47 if (client.available()) {
48   String msg = client.readStringUntil('\n');
49   if (msg.length() > 0) {
50     msg.trim();
51     Serial.print("[SERVER] Received: ");
52     Serial.println(msg);
53   }
54 }
55 }
56
57 delay(10); // Small delay to avoid busy loop
58 }

```

#### Running Logic:

1. When not connected: Check if there's a new client connection
2. When connected:
  - Check if client has disconnected
  - Read serial input and send to client
  - Read client messages and display on serial port

#### Key Command Analysis:

- `server.available()` : **Non-blocking** check for new connections. Returns empty object immediately when no connection
- `Serial.available()` : **Non-blocking** check for serial input. Returns 1 when data exists
- `Serial.readStringUntil('\n')` : Read serial input ending with newline
- `client.println(msg)` : Send message to client via TCP
- `client.available()` : **Non-blocking** check for TCP data available
- `client.readStringUntil('\n')` : Read message sent by client
- `msg.trim()` : Remove leading/trailing whitespace characters (newlines, spaces, etc.)
- `client.stop()` : Close client connection, release resources
- `delay(10)` : Short delay to avoid CPU overload, **does not affect main loop responsiveness**

### 6.3 Client Code Analysis: ESP32\_TCP\_Client.ino



💡 Client is responsible for connecting to Server's WiFi hotspot, establishing TCP connection, and performing bidirectional chat.

#### 6.3.1 Configuration Section

```

1  #include <WiFi.h>
2
3  //===== Configuration =====
4  const char* serverSSID = "ESP32_Server";    // Server's hotspot name
5  const char* serverPassword = "12345678";    // Server's hotspot password
6  const char* serverIP = "192.168.4.1";      // Server's AP IP address
7  const uint16_t serverPort = 8080;          // TCP server port
8
9

```

```

10 // ===== Pins =====
11 #define LED_PIN 2 // Status LED pin
12
13 // ===== State =====
14 WiFiClient client;
15 bool clientConnected = false;
   unsigned long lastReconnectAttempt = 0;

```

**Running Logic:** Define Server connection info, pins, and state variables.

**Key Command Analysis:**

- `serverIP = "192.168.4.1"` : Server AP mode default IP, usually no need to modify
- `clientConnected` : Connection status flag, used to control LED and reconnect logic
- `lastReconnectAttempt` : Record last reconnect time, used to implement 3-second reconnect interval

### 6.3.2 connectToServer(): Connect to Server Function

```

1 void connectToServer(){
2   Serial.print("[CLIENT] Connecting to AP: ");
3   Serial.println(serverSSID);
4
5   WiFi.mode(WIFI_STA);
6   WiFi.begin(serverSSID, serverPassword);
7
8   // Wait for WiFi connection (timeout: 15 seconds)
9   int timeout = 0;
10  while (WiFi.status() != WL_CONNECTED && timeout < 30) {
11    delay(500);
12    Serial.print(".");
13    timeout++;
14  }
15
16  if (WiFi.status() == WL_CONNECTED) {
17    Serial.println();
18    Serial.println("[CLIENT] WiFi connected!");
19    Serial.print("[CLIENT] Client IP: ");
20    Serial.println(WiFi.localIP());
21    Serial.print("[CLIENT] Server IP: ");
22    Serial.println(serverIP);
23
24    // Connect to TCP server
25    Serial.print("[CLIENT] Connecting to TCP server...");
26    if (client.connect(serverIP, serverPort)) {
27      clientConnected = true;
28      digitalWrite(LED_PIN, HIGH);
29      Serial.println(" CONNECTED!");
30      Serial.println("[CLIENT] Ready to chat. Type a message and press Enter.");
31
32      // Read welcome message from server
33      delay(500);
34      while (client.available()) {
35        String line = client.readStringUntil('\n');
36        if (line.length() > 0) {
37          Serial.print("[CLIENT] Server says: ");
38          Serial.println(line);
39        }
40      }
41    } else {
42      Serial.println(" FAILED!");
43      clientConnected = false;
44    }
45  } else {
46    Serial.println();
47    Serial.println("[CLIENT] WiFi connection FAILED. Will retry...");
48  }
49 }

```

**Running Logic:**

1. Switch to STA mode, connect to Server hotspot
2. Wait for WiFi connection (max 15 seconds)

3. After WiFi connects, connect to TCP server
4. After successful connection, read Server's welcome message

[Back to Elegoo](#)

#### Key Command Analysis:

- `WiFi.mode(WIFI_STA)` : Switch to STA mode, connect to hotspot as client
- `WiFi.begin(serverSSID, serverPassword)` : Start connecting to specified WiFi hotspot
- `timeout < 30` : Timeout counter,  $30 \times 0.5$  seconds = 15 seconds. **Timeout mechanism prevents infinite loop**
- `client.connect(serverIP, serverPort)` : Establish TCP connection. Returns `true` for success
- `client.available()` : Check if data can be read, used to read welcome message

#### 6.3.3 setup(): Initialization Function

```
1 void setup() {
2   Serial.begin(115200);
3   delay(1000);
4
5   pinMode(LED_PIN, OUTPUT);
6   digitalWrite(LED_PIN, LOW);
7
8   Serial.println();
9   Serial.println("=====");
10  Serial.println(" ESP32 TCP Client — Starting up...");
11  Serial.println("=====");
12
13  connectToServer();
14  lastReconnectAttempt = millis();
15 }
```

#### Running Logic:

1. Initialize serial port and LED
2. Print startup information
3. Immediately attempt to connect to Server
4. Record initial timestamp

#### Key Command Analysis:

- `connectToServer()` : Connect immediately at startup, don't wait for loop()
- `lastReconnectAttempt = millis()` : Initialize reconnect timing start point

#### 6.3.4 loop(): Main Loop (bidirectional chat handling)

```
1 void loop() {
2   // Check if still connected
3   if (!clientConnected || !client.connected()) {
4     clientConnected = false;
5     digitalWrite(LED_PIN, LOW);
6
7     // Attempt reconnect every 3 seconds
8     if (millis() - lastReconnectAttempt > 3000) {
9       lastReconnectAttempt = millis();
10      Serial.println("[CLIENT] Attempting to reconnect...");
11      client.stop();
12      WiFi.disconnect();
13      connectToServer();
14    }
15    return;
16  }
17
18  // 1. Read from serial input and send to server
19  if (Serial.available()) {
20    String msg = Serial.readStringUntil('\n');
21    msg.trim();
22    if (msg.length() > 0) {
23      client.println(msg);
24      Serial.print("[CLIENT] Sent: ");
25      Serial.println(msg);
26    }
27  }
28
29 }
```

```

29 // 2. Read from server and display
30 if (client.available()) {
31   String msg = client.readStringUntil('\n');
32   if (msg.length() > 0) {
33     msg.trim();
34     Serial.print("[CLIENT] Received: ");
35     Serial.println(msg);
36   }
37 }
38
39 delay(10); // Small delay to avoid busy loop
40 }

```

[Back to Elegoo](#)

#### Running Logic:

1. Check connection status, attempt to reconnect every 3 seconds when disconnected
2. Read serial input and send to server
3. Read server messages and display on serial port

#### Key Command Analysis:

- ▶ `!clientConnected || !client.connected()` : Double check connection status
- ▶ `millis() - lastReconnectAttempt > 3000` : Reconnect interval 3 seconds
- ▶ `Serial.available()` : **Non-blocking** check for serial input
- ▶ `Serial.readStringUntil('\n')` : Read serial input ending with newline
- ▶ `client.println(msg)` : Send message to server via TCP
- ▶ `client.available()` : **Non-blocking** check for TCP data available
- ▶ `client.readStringUntil('\n')` : Read message sent by server
- ▶ `msg.trim()` : Remove leading/trailing whitespace characters
- ▶ `return` : Return immediately when disconnected, don't execute subsequent code

### 6.4 Customization Guide

#### 6.4.1 Modify Hotspot Name and Password

Server Side ( [ESP32\\_TCP\\_Server.ino](#) ):

```

1 | const char* ap_ssid = "Your Hotspot Name";
2 | const char* ap_password = "Your Password"; // min 8 chars

```

Client Side ( [ESP32\\_TCP\\_Client.ino](#) ):

```

1 | const char* serverSSID = "Your Hotspot Name";
2 | const char* serverPassword = "Your Password";

```



**⚠ Both sides must be modified simultaneously!**

#### 6.4.2 Modify TCP Port

Keep the `serverPort` variable consistent on both sides. Common port numbers:

- ▶ **8080** (default) - Common HTTP alternate port
- ▶ **80** - HTTP default port
- ▶ **8888** - Another common port



**💡 Port Range:** 1~65535, but recommended to use ports above 1024 (1~1023 are system reserved ports).

#### 6.4.3 Modify Reconnect Interval

Find in [ESP32\\_TCP\\_Client.ino](#) :

```

1 | if (millis() - lastReconnectAttempt > 3000) { // 3000ms = 3 seconds

```

Change to your desired reconnect interval (in milliseconds), e.g.:

[Back to Elegoo](#)

- [1000](#) = 1 second (faster reconnection)
- [5000](#) = 5 seconds (slower reconnection)
- [10000](#) = 10 seconds

[← IOT](#)

[Bluetooth-web→](#)